DebtAware Beyond LastRunTracking? A Proxy Feasibility Boundary Study of Typed Rerun Suppression in LLVM

Anonymous Author(s)

March 23, 2026

Abstract

LLVM already has a practical nearby baseline for suppressing redundant transform-pass reruns: the 2024 *LastRunTracking* direction skips a repeated pass when no compatible intervening change has occurred. That sharply narrows the remaining research question. Starting from a proposal for *DebtAware*, we study a smaller question: for repeated cleanup passes inside a fixed stock proxy of `-O3`, do typed summaries of intervening mutations add useful signal beyond this binary heuristic when deciding whether a scheduled rerun can defer to its next stock sweep point? The executed workspace does not contain the planned patched-LLVM implementation; instead it contains a shared Python proxy harness over system LLVM. We therefore frame the result as evidence about feasibility and label identifiability, not as validation of a new compiler method. Across 9 benchmarks from CTMark, cBench, and PolyBench/C, LASTRUNTRACKING reduced compile time by 13.29% versus stock, while a preregistered RULEGUARDRAIL fallback achieved 12.82% and an aggressive RUNONCE anchor achieved 18.93% with a much higher debt-event rate. The key negative result is that all 54 sampled replay labels were debt$=1$, so every fold triggered the preregistered low-label fallback and no learned policy could be trained. The main contribution is therefore a boundary result: in this small proxy corpus, the limiting factor is not model class but the absence of class-diverse debt labels beyond what LASTRUNTRACKING already captures.

1 Introduction

LLVM optimization pipelines intentionally repeat cleanup passes because earlier transformations expose new simplification opportunities. Those reruns are useful, but they also consume compile time. The broader literature has therefore explored pass-order search, compiler autotuning, and learned pass selection (Ashouri et al., 2018; Hellsten et al., 2023; Liang et al., 2023; Liu et al., 2024; Zhao et al., 2025). For the present question, however, the strongest baseline is not a search method but LLVM's own engineering path: the October 10, 2024 RFC on avoiding transform-pass reruns that have just been run, together with the landed *LastRunTracking* analysis, already suppresses repeated executions when no compatible intervening change has occurred (dtcxzyw, 2024; LLVM Project, 2024).

That baseline lowers the novelty ceiling substantially. The remaining question is narrow: if LLVM already knows whether any compatible change happened, does slightly richer mutation-typed bookkeeping add incremental value when deciding whether a repeated cleanup pass should run now or wait until the next stock occurrence of the same pass?

The original proposal framed this as *DebtAware*: a bounded empirical test of deferred-rerun *debt* beyond LASTRUNTRACKING. The intended implementation was a patched LLVM study with counterfactual replay labels and tiny interpretable policies. The executed workspace is narrower. It contains a shared Python proxy harness over system LLVM, a fixed cleanup-heavy proxy pipeline, and four completed policies: stock, LASTRUNTRACKING, RULEGUARDRAIL, and RUNONCE. The preregistered learned policies and ablations were skipped because the replay labels collapsed completely.

This paper is therefore deliberately scoped as a *proxy feasibility boundary study*. It contributes evidence about whether the intended comparison can even be instantiated in a small CPU-only pilot, not a validated compiler optimization method.

Our contributions are:

- We refine the original DebtAware proposal into an honest proxy-study framing that isolates the incremental question beyond LLVM's LASTRUNTRACKING heuristic rather than claiming a new pass-scheduling framework.
- We show on 9 executed benchmarks that proxy LASTRUNTRACKING reduces compile time by 13.29% versus stock, while the preregistered rule fallback reaches 12.82% and the aggressive RUNONCE anchor reaches 18.93% with a 0.6667 debt-event rate.
- We identify the central feasibility bottleneck: all 54 sampled replay labels were debt$=1$, so every fold violated the preregistered minimum needed to fit learned policies.

Section 2 separates peer-reviewed related work from LLVM engineering artifacts. Section 3 describes the intended DebtAware method and the executed proxy deviation. Section 4 reports the executed evidence. Section 5 provides artifact traceability. Section 6 discusses what the negative result means.

2 Related Work

Peer-reviewed work on compiler autotuning and phase ordering. Compiler autotuning has long studied how to search or predict profitable optimization choices (Ashouri et al., 2018). Representative peer-reviewed examples include Bayesian optimization in BaCO (Hellsten et al., 2023), coreset-based pass-order prediction (Liang et al., 2023), pass-dependence modeling in DCO (Liu et al., 2024), and compilation-statistics-guided phase ordering (Zhao et al., 2025). These papers optimize over larger search spaces than we do. They search or score sequences, subsequences, or compiler configurations; our study keeps the proxy pipeline fixed and asks only whether one already-scheduled repeated cleanup pass should execute immediately or defer to the next stock sweep point.

Peer-reviewed work on pass skipping. The closest archival paper is Jayatilaka et al. (2021), which predicts whether scheduled LLVM optimizations will modify IR and uses those predictions to reduce compile time. DebtAware was explicitly proposed as a narrower follow-on question. Instead of predicting immediate IR change, it targets *deferred-rerun debt*: whether suppressing one current rerun until the next stock sweep point causes downstream divergence or unrecovered quality loss.

Compile-time measurement as a systems concern. Compile-time engineering should be evaluated directly, not only through final program runtime. Engelke and Schwarz (2024) make that point in the context of query-compilation backends. We follow the same perspective by reporting compile-time reduction, bookkeeping overhead, debt-event rate, runtime ratio on a runnable subset, and the pilot overheads that forced the study to narrow.

LLVM engineering artifacts. The direct systems baseline comes from LLVM itself rather than from a peer-reviewed paper. The October 10, 2024 LLVM RFC on avoiding reruns of transform passes that have just been run and the corresponding LASTRUNTRACKING analysis documentation show that LLVM already tracks whether compatible intervening changes have occurred since a pass last ran (dtcxzyw, 2024; LLVM Project, 2024). These sources are engineering artifacts, not archival publications, and we treat them that way. They matter here because the paper's central question is incremental: whether *typed* mutation summaries add value after that binary heuristic is already available.

Unlike the peer-reviewed pass-ordering literature, and unlike LLVM's own rerun-suppression engineering baseline, our executed artifact does not establish a new optimization method. It provides evidence about feasibility and label identifiability for that incremental question.

3 Method

3.1 Intended DebtAware method

The proposal asked whether mutation-typed summaries of intervening change improve rerun-deferral decisions beyond LASTRUNTRACKING for a very small set of repeatedly invoked cleanup passes, primarily `instcombine` and `simplifycfg`. The intended method used bounded single-rerun counterfactual replay. For a repeated pass opportunity, suppress only that immediate rerun, force execution at the next stock sweep point of the same pass kind, and label

the opportunity debt= 1 if the deferral causes post-sweep IR divergence, correctness failure, or artifact drift relative to stock `-O3`. Otherwise label it debt= 0.

The intended learned policies were tiny interpretable models, trained only if each fold and target pass had at least 24 labeled opportunities total and at least 6 examples in each class. This low-label rule was preregistered because data sparsity was a known risk.

3.2 Executed proxy study

The workspace did not execute the planned patched LLVM build. Instead, the completed artifact is a shared Python harness over system LLVM 18 tools, with a fixed proxy cleanup pipeline:

$$\texttt{mem2reg} \rightarrow \texttt{sroa} \rightarrow \texttt{instcombine} \rightarrow \texttt{simplifycfg} \rightarrow$$
$$\texttt{jump-threading} \rightarrow \texttt{instcombine} \rightarrow \texttt{correlated-propagation} \rightarrow \texttt{simplifycfg} \rightarrow$$
$$\texttt{reassociate} \rightarrow \texttt{gvn} \rightarrow \texttt{instcombine} \rightarrow \texttt{adce} \rightarrow \texttt{simplifycfg} \rightarrow \texttt{instcombine}.$$

The executed policies are:

- **Stock**: execute every scheduled pass occurrence.
- **LastRunTracking**: suppress a repeated target pass only when the binary compatible-change heuristic allows it.
- **RuleGuardrail**: a preregistered deterministic fallback layered on top of LastRunTracking.
- **RunOnce**: an aggressive anchor that executes only the first immediate repeated opportunity before deferring later occurrences to their stock sweep points.

The preregistered LearnedChange, MutationDebt, and ablations A–E were all skipped by the low-label fallback.

3.3 What the executed evidence can and cannot show

Because the artifact is a proxy harness rather than a patched compiler, we interpret it strictly as a feasibility study. A positive result would only have meant that the intended comparison looked worth pursuing in a real LLVM implementation. The executed negative result is stronger: the data never reached the point where the intended learned-policy comparison could be run at all.

4 Experiments

4.1 Setup

Benchmarks. The executed corpus contains 9 benchmarks, 3 from each suite. The benchmark list is `CTMark/{branchy_mix,cfg`
`cBench/{call_chain,mem_combo,pointer_walk}`, and `PolyBenchC/{loop_nest,mat_kernel,stencil_like`
The workspace notes that real source trees were fetched for CTMark, cBench v1.1, and PolyBench/C 4.2.1, with one deviation: CTMark is represented here by benchmark-defining translation units rather than full benchmark binaries.

Protocol. Evaluation uses three leave-one-suite-out folds. Compile measurements use 3 repetitions per benchmark and report medians inside each benchmark row. Runtime validation uses 10 repetitions on a runnable subset of 6 benchmarks: `branchy_mix`, `cfg_math`, `call_chain`, `pointer_walk`, `loop_nest`, and `mat_kernel`.

Pilot outcome. The pilot immediately showed that the planned patched-LLVM study was infeasible under the budget. Median stock compile times for the three pilot programs were 0.0952 s, 0.0983 s, and 0.1122 s, while traced median times rose to 0.4241 s, 0.7597 s, and 0.4321 s. That corresponds to 345.59%, 672.56%, and 285.32% trace overhead, respectively, with only 5 repeated opportunities per pilot benchmark.

Table 1: Main comparison for the executed proxy study. Every value is taken from `exp/stock/results.json`, `exp/last_run_tracking/results.json`, `exp/rule_guardrail/results.json`, and `exp/run_once/results.json`, using the arithmetic means stored in those files. Best deterministic values are in **bold**.

Method	Compile reduction (%) ↑	Suppression rate ↑	Debt-event rate ↓	Overhead (%) ↓	Runtime ratio	Correctness
Stock	0.00	0.0000	0.0000	**0.55**	1.0000	1.0000
LastRunTracking	**13.29**	0.4000	**0.2222**	0.63	0.9931	1.0000
RuleGuardrail	12.82	0.4000	**0.2222**	0.63	0.9912	1.0000
RunOnce	18.93	**0.6000**	0.6667	0.68	**0.9823**	1.0000

Table 2: Replay-label cardinalities from `results.json`. Every row violates the preregistered minimum needed to fit learned policies, so LearnedChange, MutationDebt, and ablations A–E were skipped.

Held-out suite	Target pass	Candidates	Labeled	debt= 1	debt= 0
PolyBench/C	`instcombine`	12	12	12	0
PolyBench/C	`simplifycfg`	6	6	6	0
cBench	`instcombine`	12	12	12	0
cBench	`simplifycfg`	6	6	6	0
CTMark	`instcombine`	12	12	12	0
CTMark	`simplifycfg`	6	6	6	0

Metrics. Our primary metric is compile-time reduction versus stock. We also report suppression rate, debt-event rate, bookkeeping overhead, runtime ratio, code-size ratio, and correctness pass rate. To avoid source ambiguity, Table 1 uses the arithmetic means materialized in each per-policy file `exp/<policy>/results.json`. We do not mix those rows with the top-level geometric-mean fields from `results.json`.

4.2 Main comparison

Table 1 reports the executed policies. LastRunTracking is the strongest credible deterministic policy in this proxy study: it reduces compile time by 13.29% versus stock with 0.4000 suppression rate, 0.2222 debt-event rate, 0.63% bookkeeping overhead, runtime ratio 0.9931, code-size ratio 1.0000, and correctness 1.0000.

RuleGuardrail does not improve on that baseline. Its compile-time reduction is slightly lower at 12.82%, with the same suppression and debt-event rates. The per-benchmark rows also show one clear downside: `PolyBenchC/loop_nest` becomes 7.71% slower than stock.

RunOnce demonstrates that more aggressive suppression can recover more compile time, reaching 18.93%, but it does so with a far worse 0.6667 debt-event rate. We therefore treat it as a failure anchor rather than a competitive policy.

4.3 Label collapse and skipped learning

The central empirical result is not a learned-policy win but a label collapse. Table 2 shows the replay-label cardinalities recorded in the top-level `results.json`. Each `instcombine` fold had 12 candidates and 12 labels, all debt= 1. Each `simplifycfg` fold had 6 candidates and 6 labels, again all debt= 1. No fold met the preregistered threshold for training.

This matters scientifically because it means the intended incremental comparison was not merely unconvincing; it was uninstantiable in the executed corpus. Mutation-typed features could not be tested as a classifier because there were no debt= 0 examples to learn from.

4.4 Fold-level and pass-level behavior

Figure 1 now shows the actual fold-level grouped bars requested by the plan. The relative ordering is stable across folds: RunOnce is always the most aggressive, while LastRunTracking and RuleGuardrail stay close.

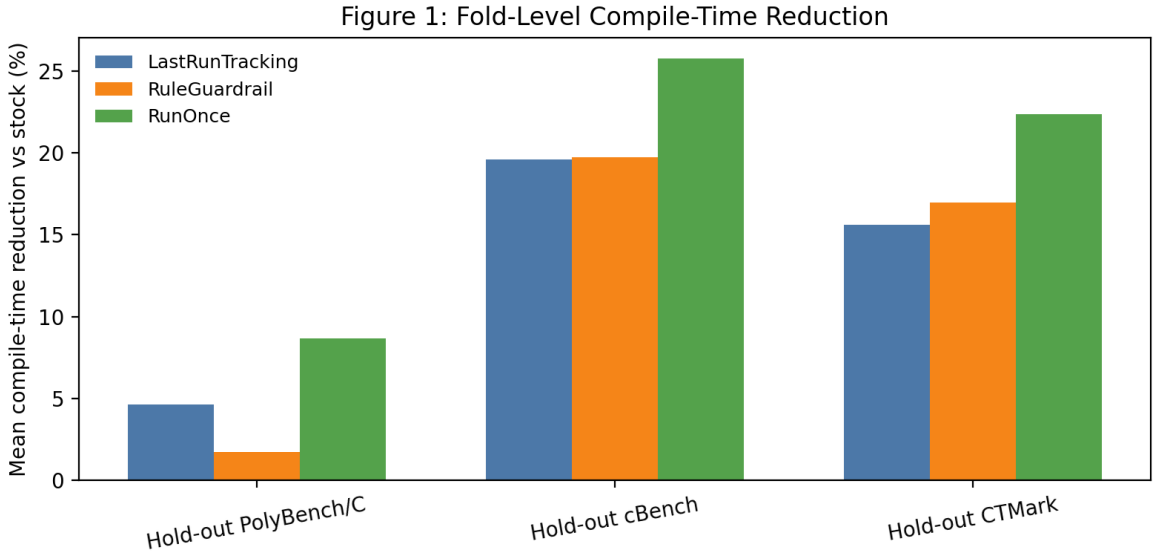

Figure 1: Mean compile-time reduction versus stock for each held-out suite. Each bar is the arithmetic mean of the 3 benchmark rows in that held-out fold, computed from the per-policy `benchmark_rows` in `exp/last_run_tracking/results.json`, `exp/rule_guardrail/results.json`, and `exp/run_once/results.json`.

Figure 2 summarizes the aggregate compile-time versus debt trade-off. Figure 3 clarifies why RUNONCE is risky: it increases the number of deferred repeated opportunities, especially for `instcombine`, and more of those deferred opportunities are associated with debt or later recovery.

5 Reproducibility and Artifact Traceability

The executed artifact is small enough that every reported result can be mapped to a specific source file.

Benchmark and pipeline provenance. The benchmark-source status is documented in `exp/original_benchmark_suites/SI` The unexecuted patched LLVM plan is documented in `exp/original_llvm_build/SKIPPED.md`. The executed proxy pipeline and policy configuration are recorded inside each per-policy `exp/<policy>/results.json`.

Executed and skipped policies. Executed policies are stock, LASTRUNTRACKING, RULEGUARDRAIL, and RUNONCE. Skipped policies are LEARNEDCHANGE, MUTATIONDEBT, and ablations A–E; each skip is recorded both in `results.json` and in `exp/<name>/SKIPPED.md`.

Metric definitions used in the paper. Compile reduction is the arithmetic mean of per-benchmark `compile_time_reduction_p` Runtime ratio is the arithmetic mean of non-null `runtime_ratio_vs_stock` values in each per-policy file. Suppression rate, debt-event rate, and bookkeeping overhead are arithmetic means over benchmark rows. Correctness is the mean `correctness_pass_rate`. Code-size ratio is 1.0000 for every executed policy.

6 Discussion and Limitations

The strongest conclusion is a boundary claim, not an algorithmic claim. This workspace does not show that DebtAware improves on LASTRUNTRACKING. It shows that, in the executed proxy corpus, the intended learned comparison cannot even be run because replay labels never produce a negative class.

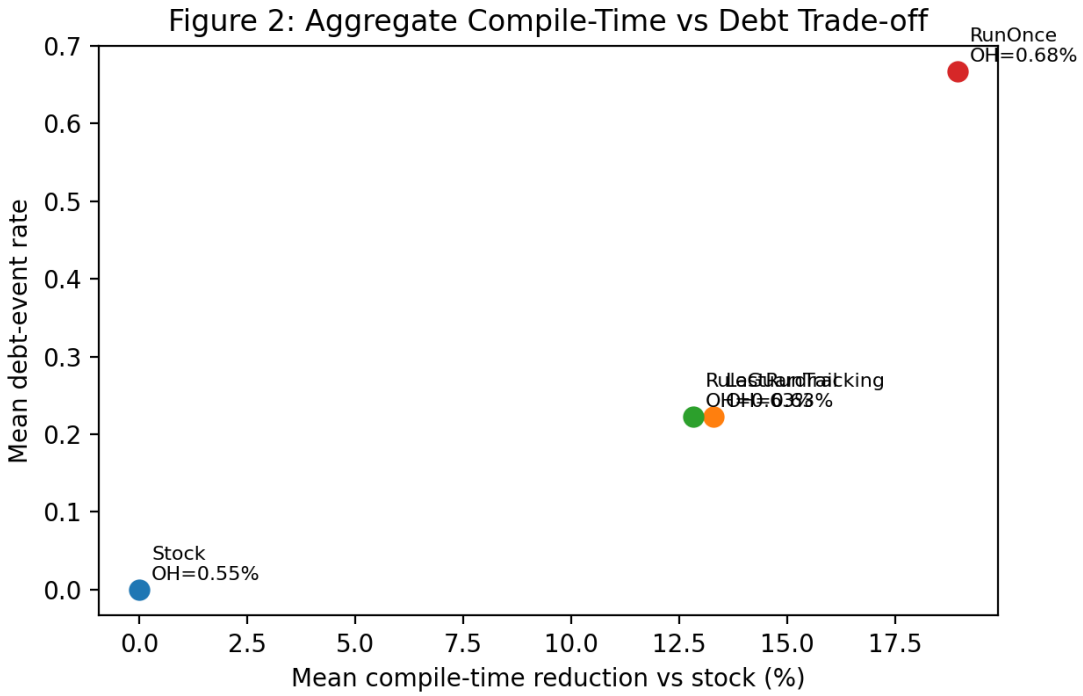

Figure 2: Aggregate trade-off between mean compile-time reduction and mean debt-event rate. Each point uses arithmetic means over the 9 benchmark rows for one policy; the annotation reports mean bookkeeping overhead.

The executed deterministic evidence is still informative. A simple binary compatible-change heuristic already recovers a substantial compile-time win in this setting: 13.29% mean reduction with no observed correctness or code-size regressions. The fixed mutation-based rule fallback does not improve on it. That weakens the immediate case for richer typed bookkeeping in this corpus, although it does not prove typed features are useless in larger or different workloads.

Several limitations remain. First, the artifact is a shared Python proxy harness over system LLVM rather than a patched compiler with one policy switch. Second, the corpus is small: 9 benchmarks, with runtime validation on only 6. Third, CTMark is represented by translation units rather than full binaries. Fourth, the pilot trace overheads were far above the planned feasibility threshold. Fifth, all skipped learned policies and ablations are skipped for a good methodological reason, but their absence still leaves the empirical story incomplete.

The next study should therefore focus less on model choice and more on data generation: a lower-overhead instrumentation path, a larger benchmark set, and a replay design that can surface naturally safe deferrals. Without those, the right claim is restraint: this artifact supplies evidence about feasibility and label identifiability, not a validated compiler optimization method.

7 Conclusion

We revisited a narrow question left open by LLVM's rerun-suppression engineering work: do typed summaries of intervening mutations add practical value beyond LastRunTracking when deciding whether a repeated cleanup rerun can wait until the next stock sweep point? In the executed workspace, the answer is a feasibility boundary rather than a positive method result. Proxy LastRunTracking reduces compile time by 13.29% versus stock, while RuleGuardrail reaches 12.82% and RunOnce reaches 18.93% with much worse debt behavior. More importantly, all 54 replay labels collapse to debt$=1$, so the preregistered low-label fallback prevents learned-policy training entirely.

The practical implication is that the next bottleneck is label identifiability, not classifier design. Before arguing for richer rerun-deferral policies beyond LLVM's existing binary heuristic, future work must first show a realistic setting

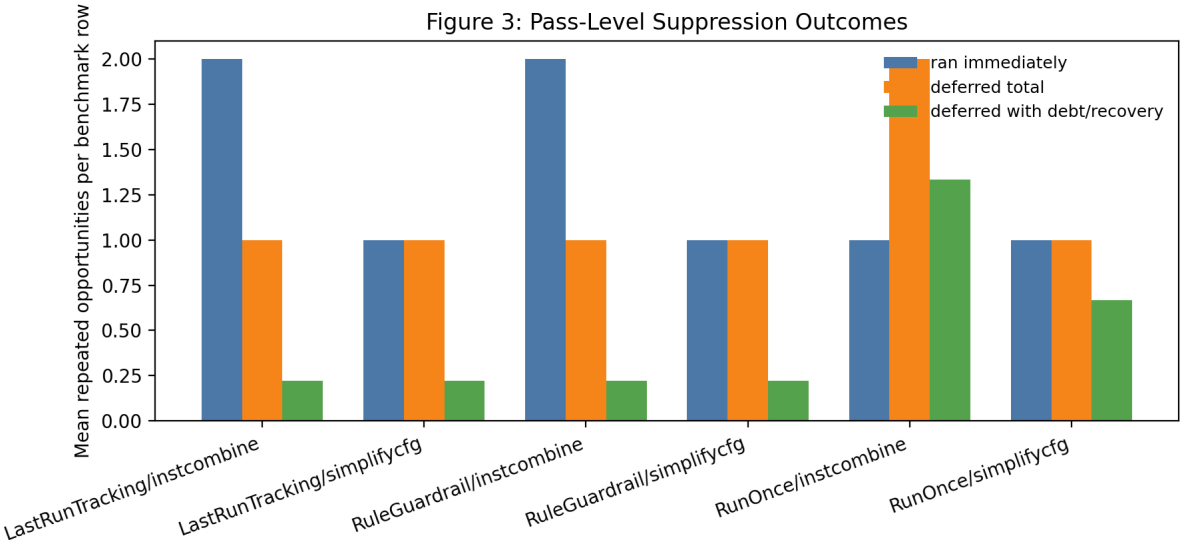

Figure 3: Pass-level grouped bars for the executed deterministic policies. The y-axis is the mean number of repeated opportunities per benchmark row. "Ran immediately" is the mean count of repeated opportunities executed at once; "deferred total" is the mean count of suppressed repeated opportunities; "deferred with debt/recovery" is the mean count of suppressed opportunities multiplied by the row's debt-event rate, so it is a subset statistic rather than an additional total.

that yields enough debt$=0$ examples to test them.

References

Amir H. Ashouri, William Killian, John Cavazos, Gianluca Palermo, and Cristina Silvano. A survey on compiler autotuning using machine learning. *CoRR*, abs/1801.04405, 2018.

Alexis Engelke and Tobias Schwarz. Compile-time analysis of compiler frameworks for query compilation. In *Proceedings of the 22nd IEEE/ACM International Symposium on Code Generation and Optimization*, pages 233–244, 2024.

Erik Hellsten, Artur Souza, Johannes Lenfers, Rubens Lacouture, Olivia Hsu, Adel Ejjeh, Fredrik Kjolstad, Michel Steuwer, Kunle Olukotun, and Luigi Nardi. BaCO: A fast and portable Bayesian compiler optimization framework. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2023.

Tarindu Jayatilaka, Hideto Ueno, Giorgis Georgakoudis, Eunjung Park, and Johannes Doerfert. Towards compile-time-reducing compiler optimization selection via machine learning. In *Proceedings of the 2021 International Conference on Parallel Processing Workshops*, pages 23:1–23:6, 2021.

Youwei Liang, Kevin Stone, Ali Shameli, Chris Cummins, Mostafa Elhoushi, Jiadong Guo, Benoit Steiner, Xiaomeng Yang, Pengtao Xie, Hugh Leather, and Yuandong Tian. Learning compiler pass orders using coreset and normalized value prediction. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 2023.

Jianfeng Liu, Jianbin Fang, Ting Wang, Jing Xie, Chun Huang, and Zheng Wang. Efficient compiler optimization by modeling passes dependence. *CCF Transactions on High Performance Computing*, 6:588–607, 2024.

LLVM Project. LastRunTrackingAnalysis.h. LLVM doxygen documentation, 2024.

dtcxzyw. [RFC][Pipeline] Avoid running transform passes that have just been run. LLVM Discourse, October 10 2024.

Table 3: Artifact map for every table and figure in this paper.

Object	Source and aggregation
Table 1	Arithmetic means stored in `exp/stock/results.json`, `exp/last_run_tracking/results.json`, `exp/rule_guardrail/results.json`, and `exp/run_once/results.json`.
Table 2	`results.json["label_cardinalities"]`.
Figure 1	Per-fold arithmetic means of `compile_time_reduction_pct_vs_stock` derived from the per-policy `benchmark_rows`.
Figure 2	Per-policy arithmetic means of `compile_time_reduction_pct_vs_stock`, `debt_event_rate`, and `bookkeeping_overhead_pct` derived from the per-policy `benchmark_rows`.
Figure 3	Per-policy arithmetic means derived from `instcombine_suppressions_mean`, `simplifycfg_suppressions_mean`, and `debt_event_rate` in the per-policy `benchmark_rows`.

Jiayu Zhao, Chunwei Xia, and Zheng Wang. Leveraging compilation statistics for compiler phase ordering. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, 2025.